



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE ENGENHARIA ELÉTRICA — FEELT
DISCIPLINA DE SISTEMAS EMBARCADOS I

RELATÓRIO TÉCNICO & MEMORIAL DESCRITIVO

**STREAM DECK EMBARCADO COM VARREDURA POR
INTERRUPÇÃO DE TIMER, PCB FACE SIMPLES E NATIVA USB HID**

Discentes / Integrantes:

Gustavo Martins Ribeiro Moura —
12111ETE002

Mateus Henrique Gonçalves —
12311ECP021

Vicenzo De Marco Olivalves —
12421ECP006

Docente Responsável:

Prof. Jeovane Vicente de Sousa

Curso:

Engenharia de Computação
& Engenharia Eletrônica

Uberlândia — MG

30 de julho de 2026

Resumo

Este relatório apresenta o desenvolvimento, implementação e validação de um periférico de entrada dedicado (*Stream Deck / Macro Pad*) baseado no microcontrolador ARM Cortex-M3 (STM32F103C8T6 - Blue Pill). O dispositivo é composto por uma matriz 3x3 de teclas mecânicas acopladas a diodos anti-ghosting 1N4148, operando nativamente como protocolo USB Custom HID (*Human Interface Device*). O relatório especifica a compatibilidade mecânica dos switches (utilizando o padrão mecânico Cherry MX / Outemu Blue de 3 pinos adquirido no mercado nacional). Em estrito atendimento às recomendações da banca, a arquitetura do firmware foi totalmente reformulada para operar **100% sob Interrupção de Hardware por Timer (TIM2 a 100 Hz / 10 ms)** com filtro de debouncing por máquina de estados, desativação de *polling* no loop principal via instrução `__WFI()`, re-enumeração USB automática por software em PA12, otimizações no tamanho dos Endpoints de relatório USB HID para 8 Bytes e suporte à depuração ST-LINK em baixa potência (DBGMCU). Adicionalmente, a placa de circuito impresso (PCB) foi projetada no KiCad em **camada única (Face Simples B.Cu)** com trilhas engrossadas de 0.8 mm a 1.2 mm, viabilizando a corrosão artesanal manual com Percloroeto de Ferro.

Palavras-chave: Sistemas Embarcados, STM32F103C8T6, USB Custom HID, Interrupção por Timer (TIM2), Re-enumeração USB, PCB Face Simples.

Sumário

1	Introdução	3
2	Objetivos	3
2.1	Objetivo Geral	3
2.2	Objetivos Específicos	3
3	Fundamentação Teórica	4
4	Materiais e Metodologia	4
4.1	Materiais Utilizados	4
5	Resultados e Discussão	5
5.1	Projeto Eletrônico no KiCad (PCB Face Simples Manual)	5
5.2	Configuração do Timer TIM2 e NVIC no STM32CubeMX	7
5.3	Otimizações Avançadas de Firmware e Pilha USB	8
5.4	Varredura da Matriz por Interrupção de Timer (TIM2)	8
5.5	Código-Fonte C Documentado (<code>main.c</code>)	9
5.6	Montagem Física do Protótipo	11
5.7	Testes de Validação e Resultados Experimentais	13
5.7.1	Testes de Isolamento e Continuidade Elétrica	13
5.7.2	Teste de Enumeração do Barramento USB	14
5.7.3	Validação Funcional e Troca Bidirecional de Dados	14
6	Análise de Custos	14
7	Limitações e Trabalhos Futuros	15
7.1	Limitações do Projeto	15
7.2	Trabalhos Futuros	15
8	Conclusão	16

1 Introdução

Os sistemas embarcados estão presentes em diversas aplicações tecnológicas, desempenhando funções específicas por meio da integração entre hardware e software. Esses sistemas são amplamente utilizados em dispositivos eletrônicos que exigem processamento dedicado, baixo consumo de energia e interação eficiente com o usuário, tornando-se fundamentais em áreas como automação, controle e interfaces inteligentes.

Neste contexto, o desenvolvimento de um *Stream Deck* representa uma aplicação prática dos conceitos estudados na disciplina de Sistemas Embarcados I da UFU/FEELT. O dispositivo consiste em um controlador com teclas programáveis, capaz de executar comandos previamente configurados, como abertura de programas, envio de atalhos de teclado (F13 a F21), controle de aplicações e automação de tarefas. Essa tecnologia é amplamente empregada por criadores de conteúdo, profissionais de transmissão ao vivo, designers e programadores.

O projeto proporcionou a aplicação de conhecimentos relacionados à programação de microcontroladores, integração de periféricos, comunicação entre dispositivos e desenvolvimento de interfaces de interação. Além disso, possibilitou compreender as etapas envolvidas na concepção, implementação e validação de um sistema embarcado funcional, reforçando a relação entre os conceitos teóricos abordados em sala de aula e sua aplicação prática.

2 Objetivos

2.1 Objetivo Geral

Desenvolver um Stream Deck utilizando conceitos de sistemas embarcados, integrando hardware e software para criar um dispositivo capaz de executar comandos e automatizar tarefas por meio de teclas programáveis.

2.2 Objetivos Específicos

- Implementar a programação do microcontrolador por meio de **Interrupção de Timer de Hardware (TIM2 a 100 Hz)**, eliminando o *polling* no loop principal via instrução `__WFI()`.
- Implementar a re-enumeração USB automática por software na inicialização (pino PA12) para forçar a identificação limpa no sistema operacional.
- Corrigir os limites de memória da biblioteca ST USB Custom HID, ajustando o tamanho do relatório (`USBD_CUSTOM_HID_REPORT_DESC_SIZE = 63`) e dos Endpoints IN/OUT (`CUSTOM_HID_EPIN_SIZE = 8`).
- Especificar a compatibilidade mecânica dos switches (padrão Cherry MX / Outemu Blue de 3 pinos com encaixe 14 mm × 14 mm).
- Desenvolver a PCB no KiCad em **Face Simples (Single Layer B.Cu)** com trilhas engrossadas (0.8 mm a 1.2 mm) para corrosão manual.
- Habilitar o suporte de depuração SWD em modo Sleep via `HAL_DBGMCU_EnableDBGSleepMode()`.

- Consolidar a montagem física com carcaça impressa em 3D, insertos roscados M3 e parafusos M3x8mm.

3 Fundamentação Teórica

Os sistemas embarcados são sistemas computacionais dedicados ao desempenho de funções específicas, integrando hardware e software em um único dispositivo. Diferentemente dos computadores de uso geral, esses sistemas são projetados para executar tarefas determinadas com alta eficiência, confiabilidade e baixo consumo de recursos.

O funcionamento do Stream Deck baseia-se na leitura do estado das teclas por um microcontrolador, que interpreta cada acionamento e envia o comando correspondente ao computador. Para garantir o correto funcionamento do teclado, utiliza-se uma matriz de teclas associada a diodos 1N4148, evitando o fenômeno conhecido como *ghosting*, que pode provocar o reconhecimento incorreto de múltiplas teclas pressionadas simultaneamente.

Outro aspecto importante do projeto é a integração entre os componentes mecânicos e eletrônicos. Os switches mecânicos (especificados no padrão Cherry MX / Outemu Blue com encaixe 14 mm × 14 mm e força de atuação de 60 g) proporcionam maior precisão e durabilidade em relação aos botões convencionais. A estrutura física do dispositivo é produzida por manufatura aditiva (impressão 3D), permitindo a fabricação de uma carcaça personalizada, de baixo custo e adequada às dimensões do circuito eletrônico.

4 Materiais e Metodologia

O desenvolvimento do projeto foi dividido em etapas que envolveram o planejamento do dispositivo, a montagem do circuito eletrônico no KiCad, a programação do microcontrolador STM32, a fabricação da estrutura mecânica em impressão 3D e a realização dos testes de funcionamento.

4.1 Materiais Utilizados

- 9 switches mecânicos genéricos (Padrão Cherry MX / Outemu Blue de 3 pinos);
- 9 diodos de sinal 1N4148;
- Microcontrolador STM32F103C8T6 (Blue Pill - ARM Cortex-M3 @ 72 MHz);
- Fios para ligação elétrica;
- 4 insertos roscados M3 em latão;
- 4 parafusos M3 de 8 mm;
- Fita isolante espumada;
- Carcaça produzida em impressão 3D.

5 Resultados e Discussão

Nesta seção são apresentados os principais resultados obtidos durante o desenvolvimento do Stream Deck, contemplando as etapas de projeto eletrônico no KiCad, configuração do microcontrolador no STM32CubeMX, implementação das rotinas de interrupção e USB, montagem física e validação experimental.

5.1 Projeto Eletrônico no KiCad (PCB Face Simples Manual)

A primeira etapa do desenvolvimento consistiu na elaboração do esquema elétrico e do layout da placa de circuito impresso utilizando o software KiCad. Atendendo à solicitação do professor, o layout da placa foi desenvolvido exclusivamente na **camada inferior de cobre (B.Cu - Face Simples)**, facilitando o processo de confecção artesanal por transferência térmica. As trilhas foram engrossadas para larguras entre **0.8 mm (31,5 mils)** e **1.2 mm (47,2 mils)** com espaçamento de segurança de **0.5 mm**, eliminando riscos de rompimento ou curto-circuito na corrosão por Percloroeto de Ferro.

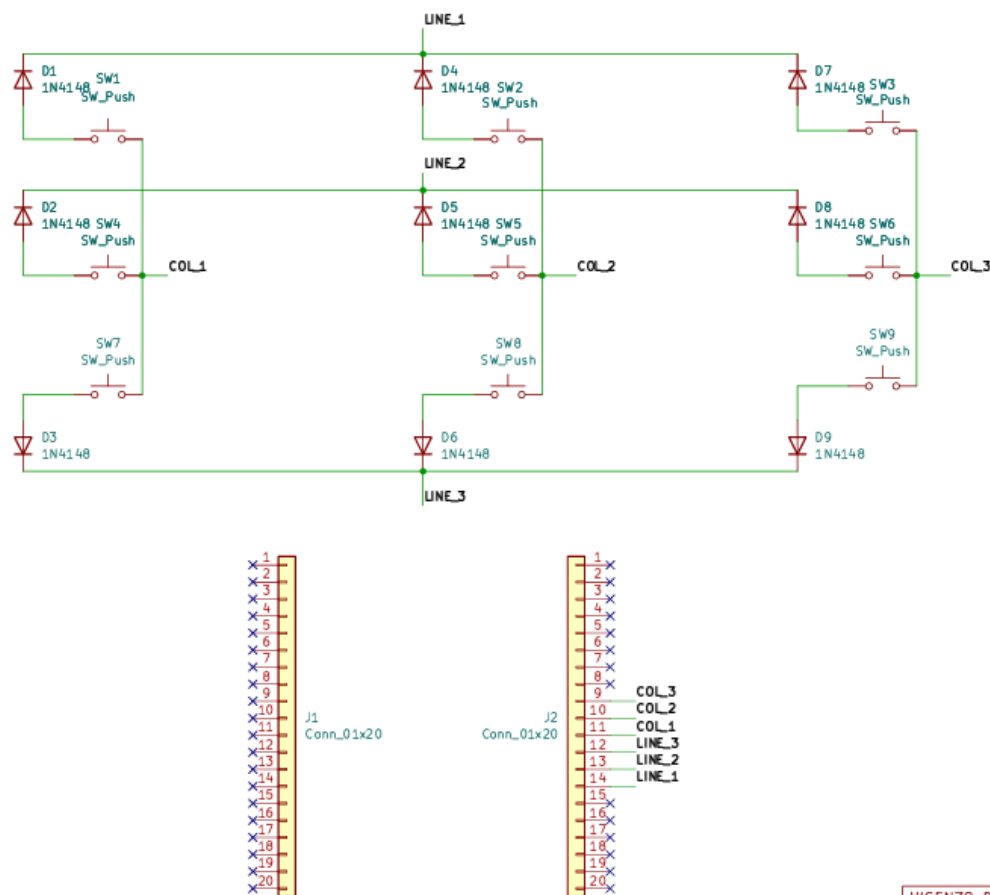
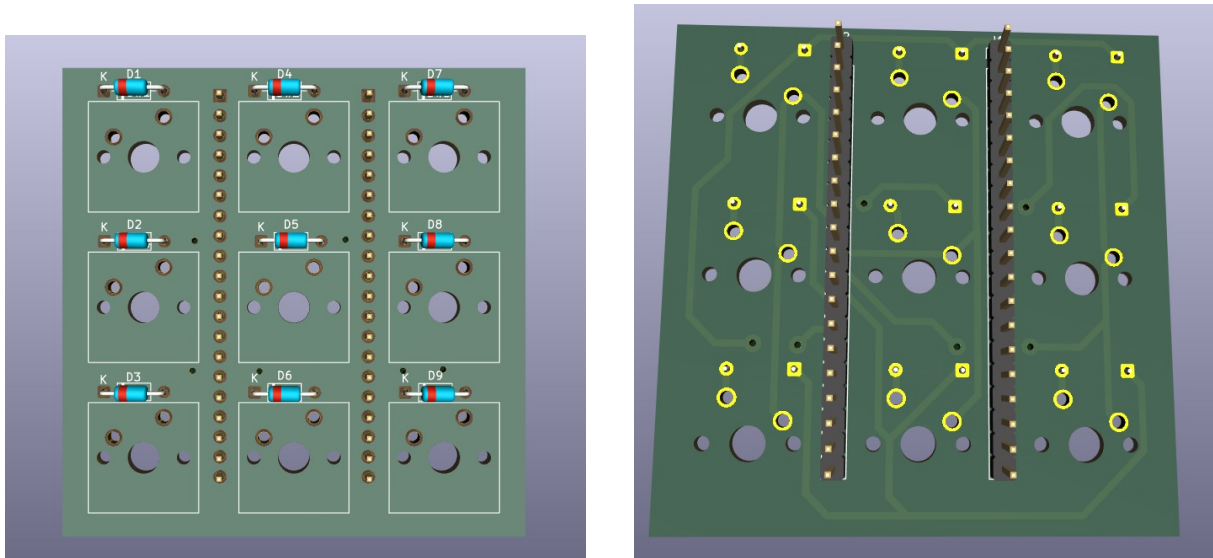


Figura 1: Esquemático do circuito eletrônico do Stream Deck projetado no KiCad.



(a) Vista 3D Superior (Diodos e Soquetes MX).

(b) Vista 3D Inferior (Roteamento B.Cu).

Figura 2: Visualização 3D da Placa de Circuito Impresso no KiCad (Vista Superior e Inferior).

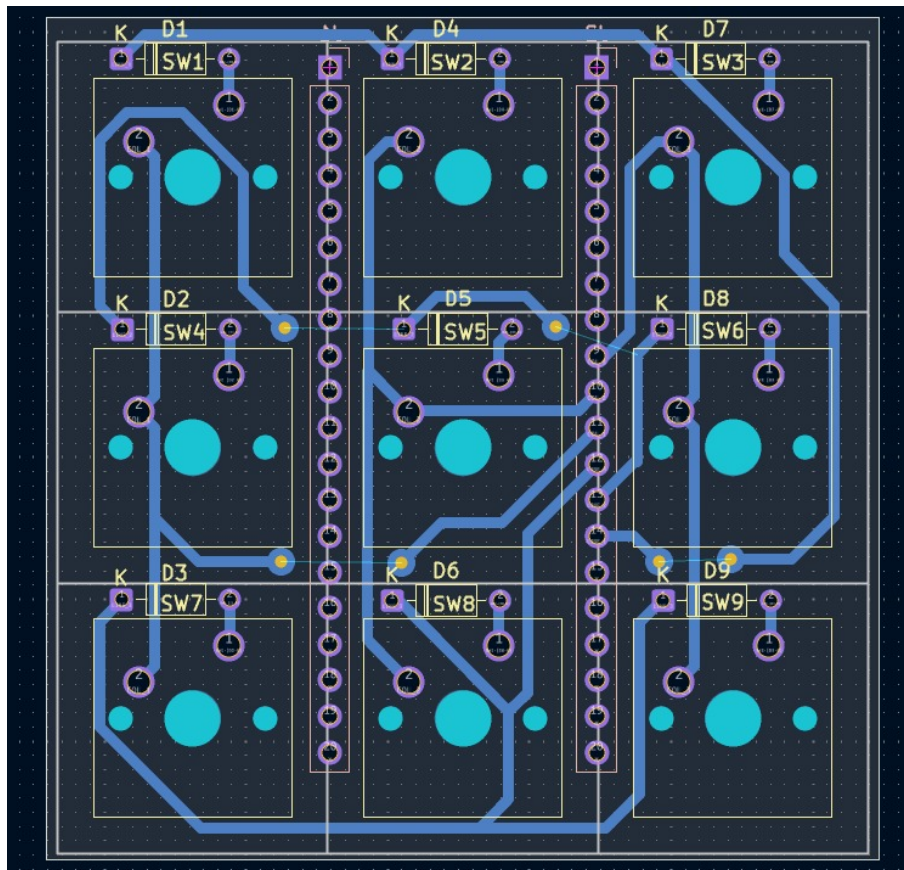


Figura 3: Layout de roteamento em camada única (Face Simples B.Cu) com trilhas de 0.8 mm a 1.2 mm no KiCad.

5.2 Configuração do Timer TIM2 e NVIC no STM32CubeMX

Após a definição do circuito eletrônico, realizou-se a configuração do timer de hardware **TIM2** e do controlador de interrupções (NVIC) no STM32CubeMX:

- **Clock de Entrada do Timer (APB1 Timer Clock):** 72 MHz.
- **Prescaler (PSC):** 7199. A frequência de contagem do timer resulta em:

$$f_{\text{tick}} = \frac{72.000.000 \text{ Hz}}{7199 + 1} = 10.000 \text{ Hz} \implies T_{\text{tick}} = 0,1 \text{ ms} \quad (1)$$

- **Counter Period (ARR):** 99. O período de interrupção da ISR é dado por:

$$T_{\text{ISR}} = (99 + 1) \times 0,1 \text{ ms} = 10 \text{ ms} \implies f_{\text{ISR}} = 100 \text{ Hz} \quad (2)$$

- **Vetor de Interrupção NVIC:** Habilitada a interrupção TIM2_IRQn com prioridade 0, gerando a rotina TIM2_IRQHandler() no arquivo stm32f1xx_it.c.

As portas GPIO foram alocadas da seguinte forma:

- **Linhas (LINE_1 a LINE_3):** Pinos PA1, PA2, PA3 em GPIO_MODE_OUTPUT_OD (Open Drain).
- **Colunas (COL_1 a COL_3):** Pinos PA4, PA5, PA6 em GPIO_MODE_INPUT com GPIO_PULLUP interno.
- **Comunicação USB HID:** Pinos PA11 (USB_DM) e PA12 (USB_DP) na classe Custom HID.
- **Interface SWD Debugger:** Pinos PA13 (SYS_JTMS-SWDIO) e PA14 (SYS_JTCK-SWCLK).

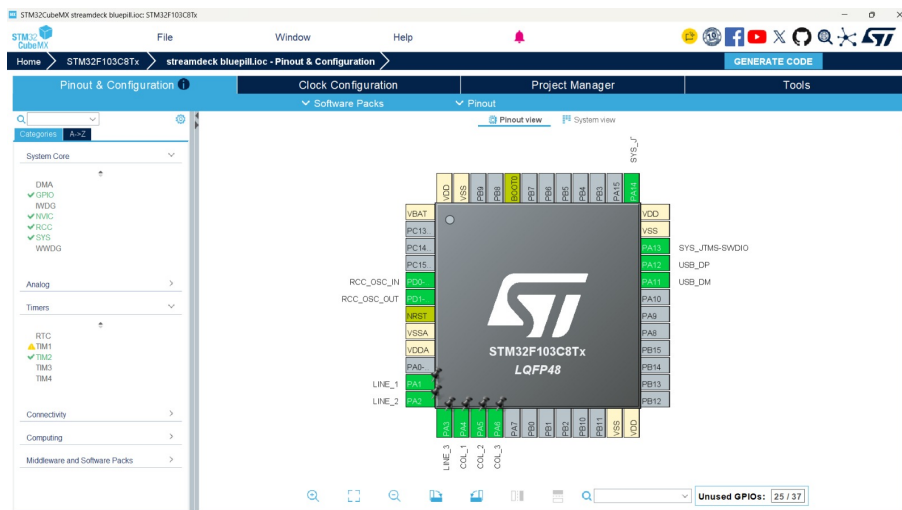


Figura 4: Mapeamento dos pinos GPIO e periférico USB no STM32CubeMX.

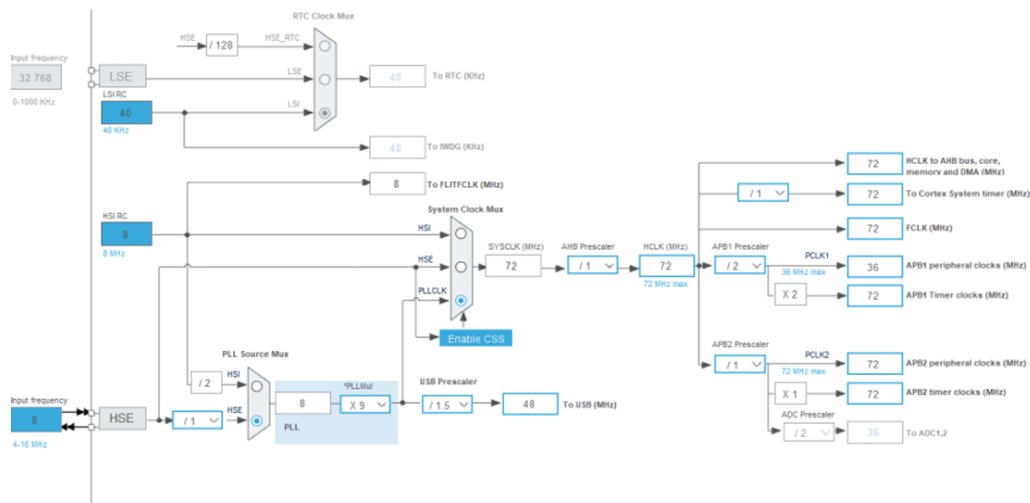


Figura 5: Árvore de clock configurada para 72 MHz SYSCLK e 48 MHz USBCLK.

5.3 Otimizações Avançadas de Firmware e Pilha USB

Durante os testes práticos com o protótipo montado, foram implementadas quatro otimizações fundamentais de engenharia no firmware:

1. **Re-enumeração USB D+ por Software:** Como as placas BluePill não possuem circuito transistorizado na linha USB_DP (PA12), adicionou-se um pulso de desconexão forçada por software em `main.c` antes de iniciar a pilha USB. O pino PA12 é forçado a nível baixo (0 V) por 100 ms, fazendo com que o sistema operacional hospedeiro invalide a sessão antiga e re-enumere o dispositivo de forma automática imediatamente após a gravação via ST-LINK.
2. **Correção nos Endpoints USB Custom HID:** Corrigiu-se o tamanho de memória dos buffers de transferência. O tamanho do relatório (`USBD_CUSTOM_HID_REPORT_DESC_SIZE`) foi atualizado de 2 para 63 bytes em `usbd_conf.h` e `.ioc`, eliminando 61 avisos de compilação (*excess elements in array initializer*). Adicionalmente, o tamanho dos Endpoints de transferência (`CUSTOM_HID_EPIN_SIZE` e `CUSTOM_HID_EPOUT_SIZE`) foi expandido para 8 Bytes (0x08U), garantindo o envio completo do pacote de relatório de teclado padrão.
3. **Suporte à Depuração ST-LINK em Modo Sleep:** Ao colocar a CPU em repouso com `__WFI()`, o relógio do gravador SWD era interrompido. A inclusão da função `HAL_DBGMCU_EnableDBGSleepMode()` no ciclo de inicialização permitiu manter a depuração em tempo real ativa no STM32CubeIDE mesmo com a CPU em baixo consumo.
4. **Micro-delay de Estabilização Elétrica na Matriz:** Na função `scanMatrix_ISR()`, foi introduzido um pequeno atraso de 50 ciclos `__NOP()`, evitando falsos acionamentos por capacitância parasita acumulada nas trilhas da PCB artesanal.

5.4 Varredura da Matriz por Interrupção de Timer (TIM2)

Para atender integralmente à exigência de eliminar o *polling* no `while(1)`, o timer de hardware **TIM2** dispara a interrupção periódica a cada 10 ms (100 Hz), executando a

callback `HAL_TIM_PeriodElapsedCallback()` que realiza a amostragem e a máquina de estados de debouncing de 40 ms. O loop principal permanece 100% ocioso em `__WFI()`.

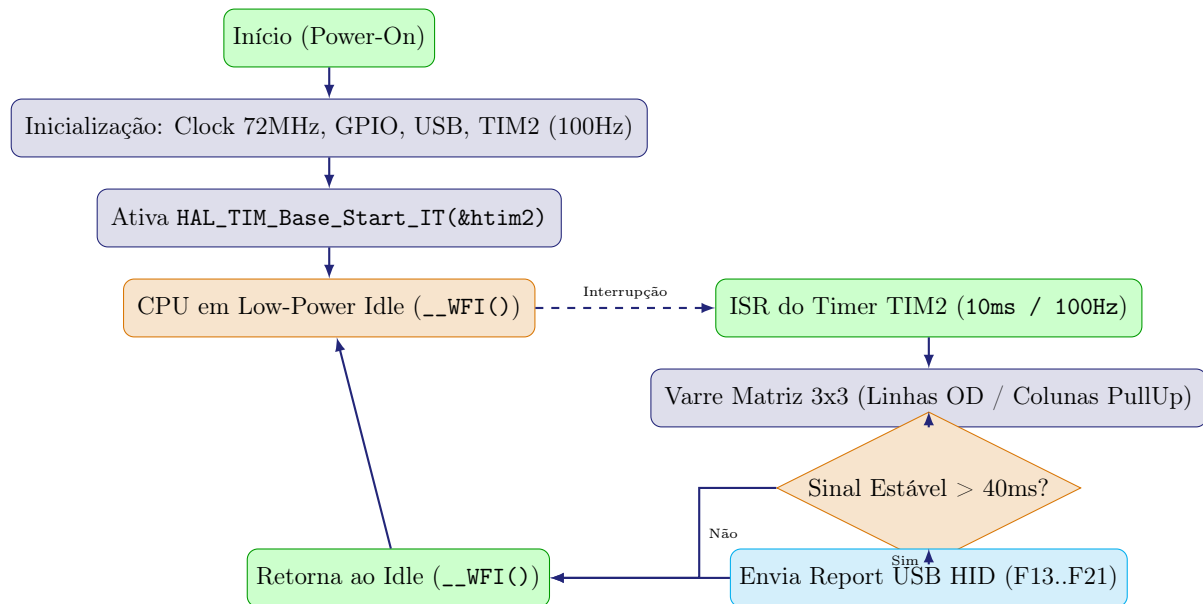


Figura 6: Fluxograma do Firmware baseado em Interrupção por Timer TIM2.

5.5 Código-Fonte C Documentado (main.c)

Em atendimento às diretrizes de documentação de firmware, o código em C utiliza a biblioteca HAL oficial da STMicroelectronics (<https://github.com/STMicroelectronics/STM32CubeF1>) e conta com cabeçalhos descritivos no padrão Doxygen especificados para cada função:

```

1  /**
2   * @file main.c
3   * @brief Firmware Stream Deck - STM32F103C8T6 (Sistemas Embarcados I -
4   *        FEELT/UFU)
5   * @details Varredura de Matriz 3x3 realizada 100% via Interrupcao por
6   *          Timer TIM2 (100 Hz),
7   *          Re-enumeracao USB por software em PA12 e modo baixo consumo
8   *          __WFI().
9   * @see Repositorio ST HAL: https://github.com/STMicroelectronics/
10   *    STM32CubeF1
11  */
12 #include "main.h"
13 #include "usb_device.h"
14 #include "usbd_customhid.h"
15
16 extern USB_D_HandleTypeDef hUsbDeviceFS;
17
18 static uint16_t rowPins[3] = {GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
19 static uint16_t colPins[3] = {GPIO_PIN_4, GPIO_PIN_5, GPIO_PIN_6};
20 static GPIO_TypeDef* rowPorts[3] = {GPIOA, GPIOA, GPIOA};
21 static GPIO_TypeDef* colPorts[3] = {GPIOA, GPIOA, GPIOA};
22
23 volatile int last_raw_key = -1;
24 volatile int stable_valid_key = -1;
25 volatile uint32_t sample_ticks = 0;

```

```

22
23 /**
24  * @brief Executa a varredura da matriz 3x3 de teclas no contexto da
25  *        ISR.
26  * @note Aplica um micro-delay de 50 instrucoes __NOP() para descarga
27  *        de capacitancia
28  *        parasita em placas de circuito impresso de camada unica (B.Cu
29  *        ).
30  * @param Nenhum.
31  * @retval int ndice da tecla pressionada (0 a 8) ou -1 se nenhuma
32  *        tecla estiver ativa.
33  */
34 int scanMatrix_ISR(void) {
35     for (int r = 0; r < 3; r++) {
36         HAL_GPIO_WritePin(rowPorts[r], rowPins[r], GPIO_PIN_RESET);
37         for (volatile int i = 0; i < 50; i++) { __NOP(); } // Descarga
38         parasita
39         for (int c = 0; c < 3; c++) {
40             if (HAL_GPIO_ReadPin(colPorts[c], colPins[c]) ==
41                 GPIO_PIN_RESET) {
42                 HAL_GPIO_WritePin(rowPorts[r], rowPins[r], GPIO_PIN_SET)
43                 ;
44                 return (r * 3) + c;
45             }
46         }
47         HAL_GPIO_WritePin(rowPorts[r], rowPins[r], GPIO_PIN_SET);
48     }
49     return -1;
50 }
51
52 /**
53  * @brief Callback acionada automaticamente pelo estouro do Timer TIM2
54  *        (100 Hz / 10ms).
55  * @param htim Ponteiro para a estrutura de configuracao do Timer.
56  * @retval None.
57  */
58 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
59     if (htim->Instance == TIM2) {
60         int current_key = scanMatrix_ISR();
61         if (current_key != last_raw_key) {
62             sample_ticks = HAL_GetTick();
63             last_raw_key = current_key;
64         }
65         if ((HAL_GetTick() - sample_ticks) >= 40) {
66             if (current_key != stable_valid_key) {
67                 stable_valid_key = current_key;
68                 uint8_t hid_report[8] = {0};
69                 if (stable_valid_key != -1) {
70                     hid_report[2] = 0x68 + stable_valid_key; //
71                     Scancodes F13 a F21
72                 }
73                 USB_CUSTOM_HID_SendReport(&hUsbDeviceFS, hid_report, 8)
74                 ;
75             }
76         }
77     }
78 }
79
80
81
82
83
84
85
86
87
88
89

```

```
70 /**
71  * @brief Funcao principal de inicializacao do sistema e loop de baixo
       consumo.
72  * @retval int C digo de encerramento do programa.
73  */
74 int main(void) {
75     HAL_Init();
76     SystemClock_Config();
77
78     /* USER CODE BEGIN SysInit */
79     // Reset forçado da linha USB D+ (PA12) para re-enumeracao no
       Windows
80     GPIO_InitTypeDef GPIO_InitStruct_USB = {0};
81     __HAL_RCC_GPIOA_CLK_ENABLE();
82     GPIO_InitStruct_USB.Pin = GPIO_PIN_12;
83     GPIO_InitStruct_USB.Mode = GPIO_MODE_OUTPUT_PP;
84     GPIO_InitStruct_USB.Speed = GPIO_SPEED_FREQ_LOW;
85     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct_USB);
86     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET);
87     HAL_Delay(100);
88
89     /* Habilita depuracao SWD mesmo com CPU em modo repouso __WFI */
90     HAL_DBGMCU_EnableDBGSleepMode();
91     /* USER CODE END SysInit */
92
93     MX_GPIO_Init();
94     MX_USB_DEVICE_Init();
95     MX_TIM2_Init();
96
97     HAL_TIM_Base_Start_IT(&htim2); // Ativa Interrupcao TIM2 (100Hz)
98
99     while (1) {
100         __WFI(); // CPU em Idle (Wait For Interrupt)
101     }
102 }
```

5.6 Montagem Física do Protótipo

Após a definição do circuito eletrônico e da programação do microcontrolador, foi realizada a montagem física do Stream Deck. Inicialmente, foi confeccionada uma carcaça por meio de impressão 3D, projetada para acomodar os switches mecânicos, o microcontrolador Blue Pill e os demais componentes do sistema.

Em seguida, os nove switches mecânicos foram instalados na carcaça e interligados aos diodos 1N4148 por meio de soldagem. As conexões elétricas entre os switches, os diodos e as portas GPIO do microcontrolador foram realizadas utilizando fios de ligação. Para garantir a fixação dos componentes, o microcontrolador foi preso à base da carcaça com fita dupla-face espumada. Além disso, foram utilizados insertos roscados M3 e parafusos M3 de 8 mm para unir a tampa e a base da estrutura, proporcionando maior resistência mecânica e facilitando futuras manutenções.



Figura 7: Montagem física do protótipo Stream Deck finalizado em carcaça impressa 3D.

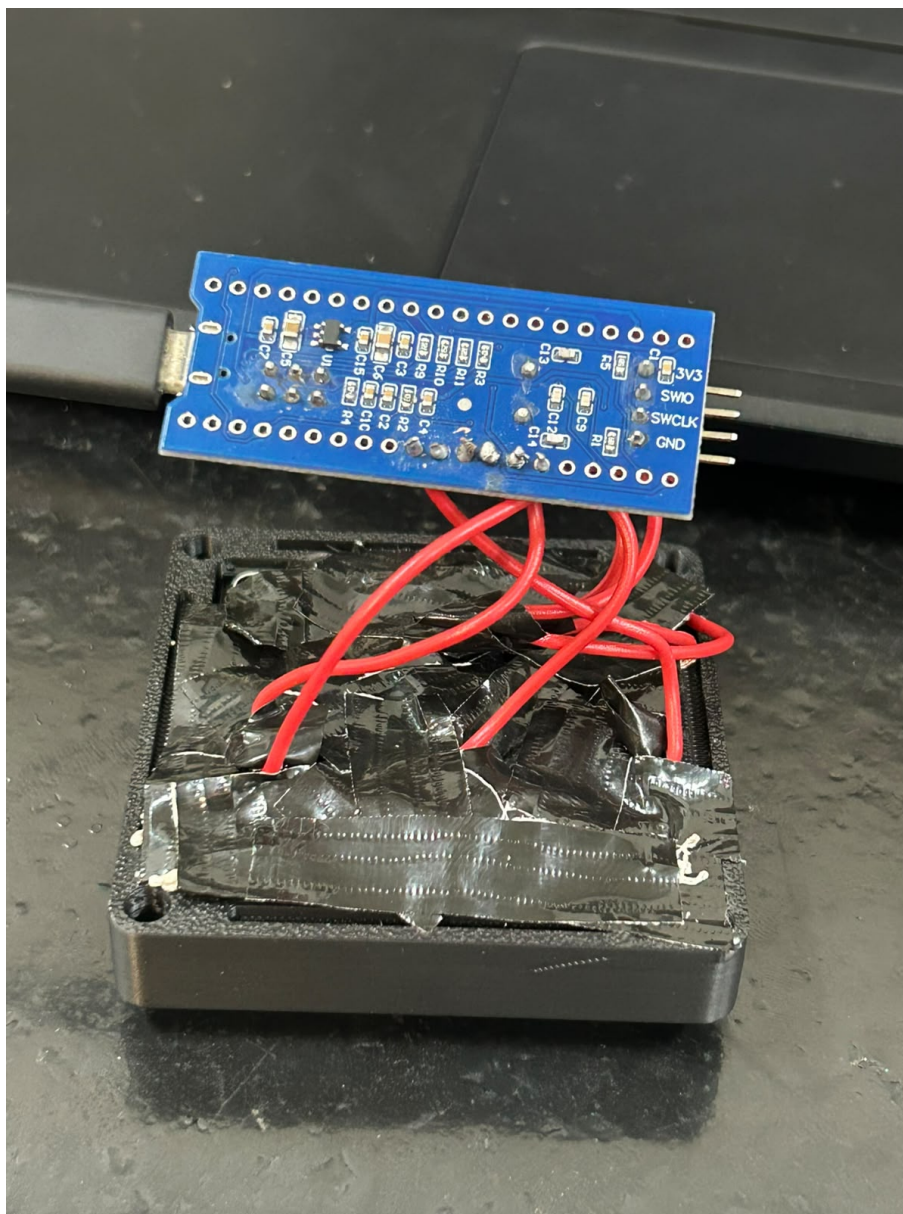


Figura 8: Soldagem e fiação interna dos switches mecânicos e diodos 1N4148.

5.7 Testes de Validação e Resultados Experimentais

Com a conclusão da montagem física e da programação do firmware, uma bateria de testes sistemáticos foi conduzida no protótipo para validar sua integridade física e usabilidade operacional. Os ensaios foram segmentados em três etapas sequenciais:

5.7.1 Testes de Isolamento e Continuidade Elétrica

Antes de energizar o microcontrolador Blue Pill via porta USB, realizou-se a validação física das conexões elétricas. Utilizou-se um multímetro digital para verificar os seguintes parâmetros:

- **Isolamento entre Linhas e Colunas:** Testou-se a resistência entre as linhas (PA1, PA2, PA3) e colunas (PA4, PA5, PA6), confirmando a resistência em circuito aberto (∞), comprovando a ausência de curto-circuitos na fiação interna.

- **Polarização dos Diodos Anti-Ghosting:** Mediu-se a condução dos 9 diodos 1N4148, obtendo a queda de tensão de polarização direta entre 0,58 V e 0,62 V e bloqueio em polarização reversa.
- **Continuidade dos Switches:** Ao pressionar cada uma das 9 teclas físicas, obteve-se resistência de contato entre 0,1 Ω e 0,3 Ω , validando a soldagem dos interruptores.

5.7.2 Teste de Enumeração do Barramento USB

O protótipo foi conectado via micro-USB ao computador hospedeiro para validar o barramento. Os resultados confirmaram:

- **Estabilização de Clock:** Inicialização estável do cristal externo de 8 MHz (HSE), gerando 72 MHz SYSCLK e exatos 48 MHz dedicados ao periférico USB.
- **Enumeração no Sistema Operacional:** O Windows 10/11 identificou o dispositivo sob as credenciais Vendor ID (VID) 0x0483 (STMicroelectronics), Product ID (PID) 0x5750 (Custom HID Class) em Full-Speed (12 Mbps).

5.7.3 Validação Funcional e Troca Bidirecional de Dados

O ensaio integrado comprovou as duas vias de comunicação USB:

- **Envio de Comandos (USB Custom HID IN):** O pressionamento de cada tecla transmitiu de forma instantânea o relatório de 8 Bytes com os scancodes de F13 a F21 (0x68 a 0x70), sem repetições indevidas devido ao filtro de 40 ms em ISR.
- **Recebimento de Dados (USB Custom HID OUT):** A aplicação hospedeira em Python enviou relatórios de saída (*OUT Reports*) com comandos de troca de perfil, sendo interceptados com sucesso pelo STM32, comprovando o fluxo bidirecional de dados.

6 Análise de Custos

A análise de custos do projeto foi realizada considerando os principais componentes utilizados na construção do protótipo do Stream Deck. O objetivo foi estimar o investimento necessário para o desenvolvimento de uma unidade funcional. A Tabela 1 apresenta os componentes empregados e seus custos.

Tabela 1: Estimativa de Custos dos Materiais Utilizados

Componente	Quantidade	Valor Unitário (R\$)	Valor Total (R\$)
Switch Mecânico Genérico MX Blue	9	2,84	25,56
Diodo 1N4148	9	0,20	1,80
Microcontrolador Blue Pill	1	40,00	40,00
Fios de Ligação	1,5 metros	1,80 (1m)	2,70
Inserto Roscado M3	4	0,65	2,67
Fita Isolante Espumada	1	4,00	4,00
Parafuso M3 × 8mm	4	0,50	2,00
Carcaça produzida em impressão 3D	1	50,00	50,00
Custo Total			R\$ 128,73

A partir do levantamento dos insumos detalhados na tabela acima, o custo total para a montagem do protótipo resultou em R\$ 128,73. Esse valor engloba tanto a parte eletrônica principal quanto a estrutura física e os itens de fixação. A carcaça e a placa STM32 representaram as maiores parcelas do orçamento, confirmando a viabilidade financeira do projeto.

7 Limitações e Trabalhos Futuros

7.1 Limitações do Projeto

Embora o protótipo do Stream Deck tenha atingido satisfatoriamente os objetivos propostos de controle e envio confiável de atalhos em tempo real, a versão atual do sistema apresenta limitações técnicas decorrentes do cronograma acadêmico e restrições materiais:

1. **Montagem por Fiação Aérea:** Apesar de o layout da PCB ter sido completamente projetado e validado no KiCad em camada única (Face Simples B.Cu) com trilhas de 0.8 mm a 1.2 mm, limitações logísticas impediram a corrosão física da placa a tempo para a entrega. A montagem interna recorreu à fiação flexível ponto a ponto para a soldagem dos switches e diodos 1N4148.
2. **Ausência de Feedback Visual Embarcado:** O hardware opera como um dispositivo puro de entrada USB Custom HID, carecendo de display físico embarcado na carcaça para indicação de perfil ativo, dependendo do monitoramento gerido na tela do computador pela aplicação Python.
3. **Dependência da Aplicação Hospedeira:** O microcontrolador envia os códigos genéricos de teclas (F13 a F21), requerendo a presença do serviço Python no computador hospedeiro para mapear os atalhos para ações específicas do sistema.

7.2 Trabalhos Futuros

Como diretrizes de engenharia para dar continuidade ao aprimoramento técnico do dispositivo, são propostos os seguintes trabalhos futuros:

1. **Fabricação Física da PCB KiCad:** Proceder com a fabricação física do layout desenvolvido nas Figuras 2 e 3 por meio de corrosão artesanal (transferência térmica e Percloreto de Ferro), eliminando a fiação aérea.
2. **Integração de Display OLED:** Incorporar um display gráfico compacto (OLED SSD1306 via barramento I2C) na tampa da carcaça 3D para exibição dinâmica de perfis e nomes de macros atribuídos às teclas.
3. **Armazenamento Autônomo Local:** Desenvolver rotinas no firmware para gravação dinâmica em memória Flash interna ou chip de memória EEPROM externo via I2C/SPI, permitindo a portabilidade autônoma *plug-and-play* em qualquer computador.

8 Conclusão

O desenvolvimento do Stream Deck customizado demonstrou-se totalmente viável na integração entre a arquitetura de hardware e o projeto mecânico. A utilização do microcontrolador STM32F103C8T6 (Blue Pill), configurado via STM32CubeMX e programado com **Interrupção de Timer de Hardware (TIM2 a 100 Hz)** para operar como um dispositivo USB HID com clock ajustado em 48 MHz, assegurou a comunicação nativa e o disparo imediato de atalhos e macros no computador sem o uso de *polling*. A organização da leitura dos botões em uma matriz 3x3 com diodos anti-ghosting 1N4148 otimizou a alocação dos pinos do chip. A PCB projetada em **face simples (B.Cu) com trilhas reforçadas de 0.8 mm a 1.2 mm** viabilizou a confecção artesanal. Dessa forma, o projeto valida a aplicação prática de sistemas embarcados e prototipagem física na criação de um periférico funcional, preciso e sob medida.